

CS780 Capstone Project: OBELIX: the Warehouse Robot

Nishant
251110053

nishantk25@iitk.ac.in

Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

We address the OBELIX robot task under partial observability, where the agent must locate, attach, and push a box to the boundary. Baseline methods (Q-learning, SARSA, DDQN) failed to learn meaningful policies and converged to rotation-dominated behavior. Our final approach combines Soft Actor-Critic (SAC) with reward shaping and controller-based exploration to handle low-information and boundary scenarios. This hybrid design significantly improves performance, demonstrating that reward design and structured exploration are more critical than model complexity in such environments.

1 Competition Result

Codabench Username: n_251110053

Final leaderboard rank (Test Phase): 87

Leaderboard rank – Level 1 (static): 61

Leaderboard rank – Level 2 (blinking): 97

Leaderboard rank – Level 3 (moving + blinking): 82

Number of valid submissions made:

Phase	Submissions
Level 1	12
Level 2	3
Level 3	2
Test	4

2 Introduction and Problem Description

The objective of this project is to design a reinforcement learning policy for the OBELIX robot to autonomously locate, attach to, and push a box to the boundary. The task involves a sequence of behaviors: find $\rightarrow$ attach $\rightarrow$ push $\rightarrow$ unwedge. The agent operates in an environment where it receives an 18-dimensional binary observation from sonar and infrared sensors and selects from 5 discrete actions.

The problem is formulated as a Partially Observable Markov Decision Process (POMDP), since the agent has access only to local sensor readings and does not observe the full state of the environment. This makes the task challenging, especially when the box is not continuously visible.

The difficulty increases across three levels: static box (Level 1), blinking box (Level 2), and moving + blinking box (Level 3). Additionally, wall obstacles introduce navigation constraints and increase ambiguity in observations. The goal is to maximize cumulative reward while achieving high success rates and ensuring robust performance across all difficulty levels and environmental conditions.

3 Approach and Model Evolution

3.1 Baseline / Initial Attempts

We began with tabular methods such as SARSA and Q-learning, followed by DDQN. While these methods were suitable for the discrete state space, they struggled in more complex settings due to limited generalization and lack of memory.

A consistent observation across these approaches was convergence to degenerate rotation-dominated policies. Due to strong penalties associated with collisions and boundaries, the agent learned to avoid forward movement and instead preferred safer rotational actions.

This highlighted the need for better exploration strategies and reward design.

3.2 Evolution Across Phases

The limitations of baseline approaches motivated the exploration of more advanced methods capable of handling partial observability and sparse rewards.

We first introduced Deep Recurrent Q-Networks (DRQN), which extend DDQN by incorporating a recurrent memory component using an LSTM. The hidden state evolves as:

$$h_t = LSTM(x_t, h_{t-1}) \quad (1)$$

$$Q(s_t, a_t) = f(h_t) \quad (2)$$

This allows the agent to retain information from past observations, which is particularly useful in scenarios where the box is not continuously visible (e.g., blinking environments).

Also, we explored Soft Actor-Critic (SAC), a policy gradient method that directly learns a stochastic policy. SAC optimizes the following objective:

$$J(\pi) = \mathbb{E}_{(s,a) \sim \pi} [Q(s, a) - \alpha \log \pi(a|s)] \quad (3)$$

The entropy term encourages exploration, which is beneficial in partially observable and sparse reward settings.

A key observation during experimentation was that **pure RL training without reward shaping consistently failed to produce meaningful behaviors**. Across multiple algorithms (Q-learning, DDQN, DRQN, and SAC), the agent often converged to rotation-dominated policies. This highlights the importance of reward design in guiding the learning process.

3.3 Final Approach and Key Design Choices

Based on extensive experimentation, we developed a hybrid framework combining:

- Soft Actor-Critic (SAC)
- Reward shaping

- Structured exploration mechanisms
- Auxiliary controllers for edge-case handling

While DRQN provided a principled solution for partial observability, SAC achieved comparable performance without requiring recurrent memory, making it a simpler and more stable choice in our setting.

3.4 Network Architecture

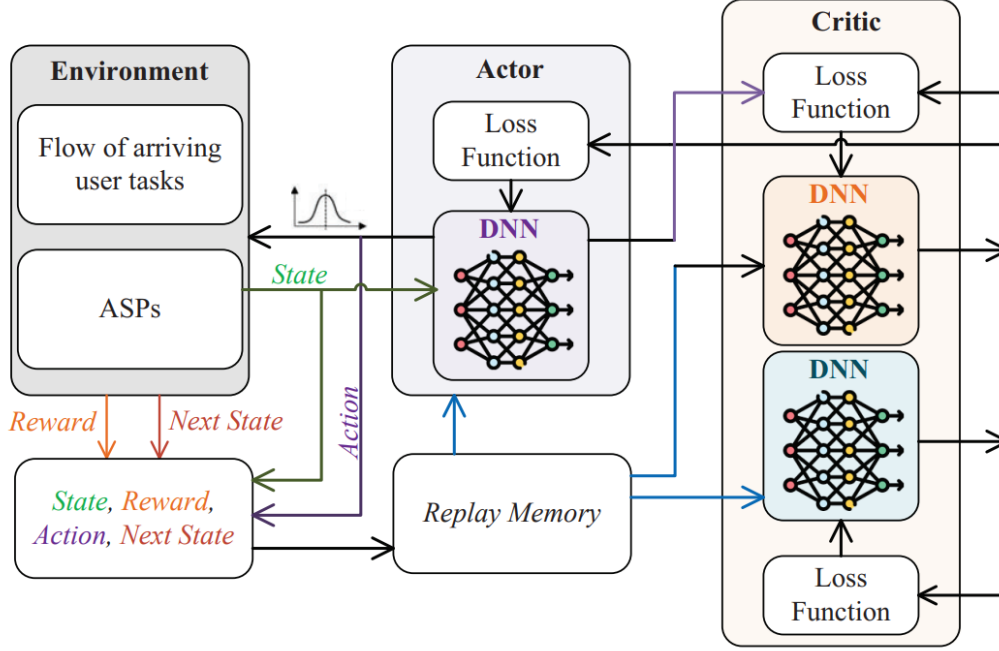


Figure 1: Soft Actor-Critic (SAC) architecture showing interaction between actor, critic networks, and replay buffer (adapted from [1]).

3.5 Exploration Strategy

Exploration is handled through a hybrid mechanism:

- Intrinsic stochasticity of the SAC policy
- Controller-based exploration in low-information scenarios

Specifically:

- When all observations are zero, a sweep-based exploration strategy is activated
- When the agent is stuck, an escape mechanism is triggered

This ensures effective exploration even when the policy lacks sufficient information.

3.6 Reward Shaping

Reward shaping was essential for stabilizing learning. The shaped reward function can be summarized as:

$$r' = -0.01 + 0.05 \cdot \mathbb{K}_{forward} + 0.1 \cdot \mathbb{K}_{sensor-active} - 0.5 \cdot \mathbb{K}_{stuck} + R_{goal} \quad (4)$$

where:

$$R_{goal} = 1000 + 200 \cdot \frac{T - t}{T} \quad (5)$$

This reward encourages forward movement, penalizes being stuck, and provides a large terminal reward for successful task completion.

4 Experiments and Implementation Details

4.1 Training Setup

The agents were trained for a total of 6000 episodes, with a maximum horizon of 2000 steps per episode. Training was performed on a CPU-only setup, and the total training time was approximately 30 hours.

To ensure stable learning and iterative improvement, training was conducted in phases. Specifically, the model was trained in batches of 2000 episodes (approximately 10 hours per batch), followed by evaluation and analysis. Based on performance, further training was continued from checkpoints. This iterative process helped refine hyperparameters and improve robustness.

4.2 Optimization and Hyperparameters

All models were trained using the Adam optimizer. For SAC, separate optimizers were used for the actor and critic networks, while DRQN used a single optimizer for the Q-network.

The key hyperparameters for both models are summarized in Table 1.

Parameter	SAC	DRQN
Learning Rate	3×10^{-4}	5×10^{-4}
Batch Size	64	32
Discount Factor (γ)	0.99	0.97
Replay Buffer Size	10000	10000
Min Buffer Size	50	—
Sequence Length	—	32
Burn-in Steps	—	8
Target Update	$\tau = 0.01$	20 steps
Entropy Coefficient (α)	0.2	—
Exploration Strategy	Stochastic Policy	ϵ -greedy
Epsilon (start $\rightarrow$ end)	—	0.75 $\rightarrow$ 0.05

Table 1: Hyperparameters used for SAC and DRQN models

4.3 Training and Evaluation Strategy

To improve generalization, a curriculum-based seed strategy was used. During early training, a limited set of seeds was used ($ep\%50$), which was gradually increased ($ep\%100$) before transitioning to fully randomized seeds. This allowed the agent to progressively adapt to diverse initial conditions.

The final model used for evaluation was primarily trained on difficulty level 2 (blinking box) without wall obstacles, as it provides a balanced level of complexity.

Evaluation was performed using the provided `evaluation.py` script over 25 randomly sampled unseen seeds. To avoid overfitting, training and evaluation seeds were kept distinct, and model selection was based on consistent performance across multiple runs rather than single-seed outcomes.

5 Results

5.1 Performance Across Models

Table 2 shows the performance of baseline methods on Level 1. All baseline methods (Q-learning, SARSA, DDQN) achieved similar performance, indicating limited improvement from value-based approaches without additional enhancements.

Model	With Wall	No Wall
Q-learning / SARSA / DDQN	−1981.2	−1990.9

Table 2: Baseline performance on Level 1

5.2 Development Phase Results

Level	With Wall (Mean $\pm$ Std)	No Wall (Mean $\pm$ Std)	Weighted Reward
Level 1	−1981.2 $\pm$ 7.7	−1990.9 $\pm$ 10.4	−
Level 2	−39445.9 $\pm$ 52873.8	−7478.9 $\pm$ 7134.3	−
Level 3	−6308.9 $\pm$ 4044.4	−4042.1 $\pm$ 3167.0	−5402.2

Table 3: Performance across different difficulty levels

5.3 Final Test Phase Results

Table 4 summarizes the performance of DRQN and SAC across all difficulty levels.

Model	Difficulty	With Wall	No Wall
DRQN	Level 1	−17661.0	−2085.7
	Level 2	−17909.3	−3612.5
	Level 3	−6308.9	−4042.1
SAC	Level 1	−16153.6	−1110.9
	Level 2	−17232.4	−3642.9
	Level 3	−10567.7	−3606.8

Table 4: Final test phase performance across difficulty levels

5.4 Overall Performance

Model	With Wall	No Wall
DRQN	-13959.7	-3246.8
SAC	-14651.2	-2786.9

Table 5: Overall average performance

The weighted cumulative reward was -9674.5 for DRQN and -9905.5 for SAC.

5.5 Analysis

- DRQN and SAC significantly outperform baseline methods, highlighting the importance of reward shaping and structured exploration. Baseline approaches consistently converged to rotation-dominated policies and failed to generalize beyond simple scenarios.
- In terms of performance, the SAC agent achieves strong success rates across different settings: approximately 100% success on evaluation over 10 random seeds in static environments, around 80% in blinking scenarios (Level 2), and close to 100% in moving + blinking environments (Level 3).
- DRQN achieves slightly better overall performance, particularly in wall environments.
- SAC performs competitively and shows better performance in no-wall settings, demonstrating that effective policies can be learned even without explicit memory when combined with reward shaping and exploration strategies.
- Overall, performance is primarily driven by reward design and structured exploration, while model complexity (e.g., memory vs non-memory methods) plays a secondary role.

6 Error Analysis and Discussion

6.1 Failure Modes

- **Level 1 (Static Box):** Failures mainly arise due to wall interference. Since walls trigger similar sensor patterns as the box, the agent may misinterpret obstacles as targets, leading to unnecessary rotations instead of forward movement.
- **Level 2 (Blinking Box):** Blinking introduces temporal inconsistency. The agent may move forward when the box is visible but overshoot when it disappears, resulting in loss of alignment and difficulty in pushing the box.
- **Level 3 (Moving + Blinking Box):** In addition to blinking and wall ambiguity, the moving target introduces interception challenges, making consistent tracking harder.
- **With Walls:** As noted above, the similarity in sensor activations between walls and the box leads to incorrect decisions and inefficient navigation near boundaries.

6.2 Limitations and Discussion

- A key limitation arises from ambiguous observations, where walls and the box produce indistinguishable sensor patterns, as discussed in the failure modes.
- The agent can exhibit oscillatory behavior (e.g., repeated left-right rotations or moving around the openings) when alternating sensor activations occur. Since this does not correspond to a “stuck” condition, recovery mechanisms are not triggered.
- While reward shaping and controllers improve robustness, they do not fully resolve these ambiguities.
- Memory-based approaches (e.g., DRQN) did not significantly mitigate these issues, suggesting that more expressive representations may be required.

6.3 Future Directions

- Incorporating richer state representations or belief-based methods to reduce ambiguity
- Using more expressive memory architectures
- Applying preference-based learning (e.g., RLHF) to discourage oscillatory behaviors

7 Conclusion

- Developed a hybrid RL framework combining SAC with reward shaping and controller-based exploration, achieving strong performance across difficulty levels.
- Demonstrated that reward design and structured exploration have a greater impact than model complexity in partially observable environments.
- Highlighted the importance of handling edge cases (e.g., no observations, boundary interactions) for robust performance.
- Extending the approach to more realistic environments and multi-object scenarios remains a promising direction.

8 Code and Demonstration Link

The implementation and additional resources for this project are provided below:

- **GitHub Repository:**
<https://github.com/Nishant-ml/CS780-Project>
- **Project Demonstration Video:**
<https://youtu.be/Fz3YwjtmdY0>

LLM Usage Declaration

- Which Large Language Model (LLM) did you use to complete the competition or write the report?
Chatgpt

- In which part did you use the LLM, and how did it help you?

Part	Used LLM? (Yes/No)	Purpose of Using LLM
Understanding concepts	Yes	Clarifying rl concepts
Ideation / brainstorming	Yes	for refining the algorithmic ideas
Debugging small code snippets	Yes	identifying and fixing code issues
Plotting / visualization code	No	–
Grammar / writing style	Yes	improving readability of report
Report structure / section ideas	No	–
Other (specify)	No	–

- Did you refer to any other sources or websites apart from the LLM and lecture slides?

No

If yes, please mention clearly (papers, blogs, GitHub repos, textbooks etc.)

- Apart from any LLM usage, what was your own contribution?

Briefly describe your personal understanding, reasoning, implementation effort, analysis, and experimentation.

My contributions include brainstorming algorithmic ideas, implementing different reinforcement learning approaches, performing hyperparameter tuning, and conducting experiments and evaluation of results.

References

- [1] Du, X. (2023). Model Architecture of Soft Actor-Critic Networks.